MECHATRONICS

Course Project

Blind Assist Device

Jonathan Damtoft Engell (202206183)
Erik Youden Munkholm (202206099)
Alessandro Gorla (202400715)
AanandNataraj Satheesh Manimala (202403949)

Table of contents:

Introduction

With advancements in technology, assistive devices have become increasingly capable of empowering individuals with disabilities, providing them with greater independence and improved quality of life. One such application is the development of innovative solutions for the visually impaired, addressing the challenges they face in navigating their environments.

This project focuses on creating a blind assist device that integrates soft actuators and solenoid valves controlled through an Arduino-based system. Soft actuators, known for their flexibility and adaptability, are ideal for applications requiring gentle and precise interactions, making them suitable for guiding and alerting visually impaired users. Solenoid valves, in conjunction with these actuators, enable efficient control of the pneumatic system, ensuring responsive and reliable operation.

The objective of this project is to design and prototype a compact, user-friendly device that enhances situational awareness for visually impaired individuals. Leveraging the Arduino platform facilitates cost-effective development and simplifies the integration of components, allowing for customization and scalability.

This report outlines the conceptualization, design, implementation, and testing of the device, emphasizing its potential to improve the autonomy and safety of visually impaired individuals.

Requirements

- Budget of 700Kr
- Navigate blind people by detecting obstacles

Component list

S.No	Component	Quantity
01	Arduino Uno	1
02	Ultrasonic Sensor-HCSR04	2
03	Resistor - 370Ω	3
04	Transistor-TIP122G	3
05	Rectifier Diode-IN4007	3
06	DC 6V air pump	1
07	12V DC 2-position 3-way Mini Solenoid Valve for Air flow-(Model: Fa0520F)	2
08	Fabricated Silicon actuators	2
09	AA 1.5V batteries	8
10	Silicon pump	1
11	Breadboard 830 MB-102	1
12	Jumper wires	40
13	Tube splitter	1
14	9V battery	1

Table 1

Soft Actuator

Fabrication process:

Step 1: Mixing base and mixer with 1:1 ratio. Figure 1.

Figure 1

Step 2: Pouring the mixed silicone into the mold, that has been 3D printed and designed to accommodate the first layer of the soft actuator.

Step 3: Place the mold with silicone inside oven at temperature of 50 °C for 2 hours. This process makes the solidification of the silicon faster and more uniform removing the undesired bubbles. Figure 2.

Figure 2

Step 4: Pour a second thin layer on baking paper and place the solidified part on top of it to seal it. Both layers are then put in the oven to obtain the final actuator.

Step 5: Insert the tube using a needle.

Step 6: Assemble the two soft actuators with the pump and the valves using tubes following the diagram in Figure 3.

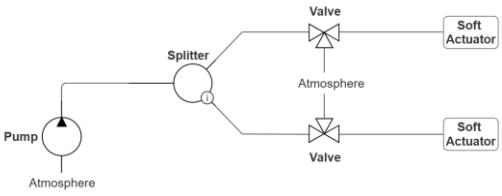

Figure 3

Control

The system is controlled by an Arduino microcontroller, which is powered by a 9V external battery, which enables it to control and run the DC-motor of the pump and open and close the valve using an external 12V battery set. This circuit can be seen in Figure 4.

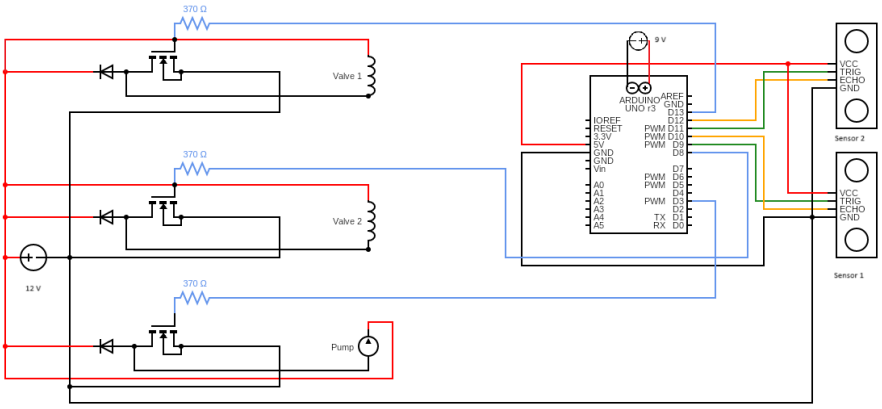

Figure 4

The code necessary to enable the circuit in Figure 4 can be split into the following sections.

Pin assignments: Establishing constants for the various components, in which each *"const int"* declares a specific pin number on the Arduino board to a specific component in the circuit.

Distance threshold: The constants "*maxDistance"* and *"minDistance"* serve as boundaries for the ultra-sonic sensors and thus the signaling and controlling the system based on the distance measurements.

Variables tracking state: In this part of the code specific variables states are defined to be tracked. The tracked variables and purpose of tracking them can be seen in table 2.

Tracked Variables	Function
`Bool motorState = false`	Tracking the state of the motor ON *(true)* or OFF *(false) (Arduino.cc, 2024).*
`unsigned long previousValve(1 or 2)Millis = 0`	Storing the last recorded action related to either solenoid valve 1 or 2 (Arduino.cc, 2024).
`Const unsigned long valve interval = 100`	Interval between opening and closing for the valves (Arduino.cc, 2024).
`Bool valve(1/2)State = false`	Current state tracked for solenoid valve(1/2) (HIGH/LOW, ON/OFF) *(Arduino.cc, 2024).*

Table 2

Void setup: Configuring the starting setup of the pins and components when the microcontroller starts up (Arduinofactory.com, 2024).

Output Pin In Void Setup	Function
`pinMode(valve(1/2), OUTPUT)(pin 13 or 8)`	The solenoid valves connected to either pin(13 or 8) are now controllable (HIGH/LOW).
`pinMode(motor, OUTPUT)(pin 3)`	The motor connected to pin 3 is now controllable (HIGH/LOW).
`pinMode(trigPin(1/2),OUTPUT)(pin 9 or 11)`	The trigger pin of both HC-SR04 is sending ultrasonic pulses, which are periodically set to HIGH to emit the ultrasonic pulses (ArduinoTech, 2024).
`pinMode(echoPin(1/2),INPUT)(pin10 or 12)`	The echo pin of both HC-SR04 is receiving reflected ultrasonic waves, which are in turn used to determine the distance of the sensed object (ArduinoTech, 2024).

Table 3

Void loop: In the void loop function repetitive operations are performed by the Arduino after the execution of the void setup (ed, 2020).

```
float distance(1/2) = measureDistance(trigPin(1/2),echoPin(1/2).
```

Sending a short pulse from the trigPin(1/2), and then measuring the duration of the signal received in echoPin(1/2), which at last converts the duration into distance relating it with the speed of sound. These distances are used to make decisions about turning ON or OFF the motor and the valves. Note the measureDistance function can be seen in the appendix.

```
if ((distance1 >= minDistance && distance1 <= maxDistance) ||
      (distance2 >= minDistance && distance2 <= maxDistance)) {
    if (!motorState) {
      // Turn on motor
```

```
      digitalWrite(motor, HIGH);
      motorState = true;
  }
```

Registers if the motor should be turned ON because at least one object is within range.

```
   } else {
     if (motorState) {
       // Turn off motor and valves
       digitalWrite(motor, LOW);
       digitalWrite(valve1, LOW);
       digitalWrite(valve2, LOW);
       motorState = false;
     }
   }
```

Else the motor is stopped, and the solenoid valves are deactivated, if no object is detected by the sensors within the range.

```
 // Control valve 1 based on Sensor 1
   if (distance1 >= minDistance && distance1 <= maxDistance) {
     unsigned long currentMillis = millis(); // Current time since the start of the Arduino board program
     if (currentMillis - previousValve1Millis >= valveInterval) {
       valve1State = !valve1State; // Toggle valve 1 state determined from the above line, which essentially
determines if enough time has passed to toggle the state of Valve
       digitalWrite(valve1, valve1State ? HIGH : LOW); // Enforcing the new state upon the valve physically
       previousValve1Millis = currentMillis; // Setting the current time as the previous time and the starting over
     }
   } else {
     digitalWrite(valve1, LOW); // Turn off valve 1 if no object detected by sensor 1
   }
```

The solenoid valve 1 is activated if sensor 1 is detecting an object, which then introduces a periodic toggling of the valve state to avoid a continuous operating state, which in this case could cause the pump to overinflate the soft actuator, and there will be no vibrational haptic feedback as the toggling of the solenoid valve leads to periodic inflation and deflation. Note that the above code and operation is identical for solenoid valve 2.

In summary two pneumatic soft actuators have been created that are controlled by two ultrasonic sensors independently, in this way it is possible to differentiate whether the obstacle is on the right or on the left of the user. The circuit used to control this system is represented in Figure 4 and executed by the code explained above.

Conclusion

The developed assistive device provides an intuitive and effective solution for visually impaired individuals to sense obstacles in their environment. The integration of ultrasonic sensing with pneumatic soft actuators creates a novel, low-cost, and practical tactile feedback system that enhances user independence and mobility. Furthermore, the application of the pneumatic soft actuator, will feel less mechanical in its tactile feedback, compared to conventional electrical vibrational devices. Further development can include waterproofing, ergonomic enhancements, faster response time, and extended sensing range for more robust usability.

Literature list

1. Arduino.cc. (2024). Available at: https://docs.arduino.cc/language-reference/en/variables/data-types/bool/ [Accessed 21 Nov. 2024].
2. Arduino.cc. (2024). Available at: https://docs.arduino.cc/language-reference/en/variables/data-types/unsignedLong/ [Accessed 21 Nov. 2024].
3. Arduinofactory.com. (2024). *Arduino language: void setup*. [online] Available at: https://arduinofactory.com/arduino-language-void-setup/ [Accessed 21 Nov. 2024].
4. ArduinoTech. (2024). *HC-SR04 Afstands Måler*. [online] Available at: https://arduinotech.dk/shop/hc-sr04-afstands-maaler/ [Accessed 12 Nov. 2024].
5. ed (2020). *Arduino Void Setup and Void Loop Functions [Explained] - The Robotics Back-End*. [online] The Robotics Back-End. Available at: https://roboticsbackend.com/arduino-setup-loop-functions-explained/ [Accessed 22 Nov. 2024].

Appendix

Code for controlling the circuit

```
// Solenoid valve 1
const int valve1 = 13;
// Solenoid valve 2
const int valve2 = 8;
// DC-motor-pump
const int motor = 3;
// HC-SR04 Sensor 1
const int trigPin1 = 9;
const int echoPin1 = 10;
// HC-SR04 Sensor 2
const int trigPin2 = 11;
const int echoPin2 = 12;

// Constants for distance thresholds
const float minDistance = 20.0;   // Minimum distance in mm (object very close)
const float maxDistance = 1000.0;  // Maximum distance in mm (object far away)

// Variables tracking state
bool motorState = false;  // Tracks if the motor is currently on
unsigned long previousValve1Millis = 0; // Valve 1 timing
unsigned long previousValve2Millis = 0; // Valve 2 timing
const unsigned long valveInterval = 100; // 0.1 second interval
bool valve1State = false;  // Tracks valve 1 state (HIGH/LOW)
bool valve2State = false;  // Tracks valve 2 state (HIGH/LOW)

void setup() {
  pinMode(valve1, OUTPUT);
  pinMode(valve2, OUTPUT);
  pinMode(motor, OUTPUT);
```

```
  pinMode(trigPin1, OUTPUT);
  pinMode(echoPin1, INPUT);

  pinMode(trigPin2, OUTPUT);
  pinMode(echoPin2, INPUT);

  Serial.begin(9600);
}

void loop() {
  // Measure distances using the two HC-SR04 sensors
  float distance1 = measureDistance(trigPin1, echoPin1);
  float distance2 = measureDistance(trigPin2, echoPin2);

  //Printing distances to Serial Monitor
  Serial.print("Distance 1: ");
  Serial.print(distance1);
  Serial.print(" mm, Distance 2: ");
  Serial.println(distance2);

  // Check if either sensor detects an object within range
  if ((distance1 >= minDistance && distance1 <= maxDistance) ||
      (distance2 >= minDistance && distance2 <= maxDistance)) {
    if (!motorState) {
      // Turn on motor
      digitalWrite(motor, HIGH);
      motorState = true;
    }
  } else {
    if (motorState) {
      // Turn off motor and valves
      digitalWrite(motor, LOW);
      digitalWrite(valve1, LOW);
      digitalWrite(valve2, LOW);
      motorState = false;
    }
  }

  // Control valve 1 based on Sensor 1
  if (distance1 >= minDistance && distance1 <= maxDistance) {
    unsigned long currentMillis = millis(); // Current time since the start of the Arduino board program
    if (currentMillis - previousValve1Millis >= valveInterval) {
      valve1State = !valve1State; // Toggle valve 1 state determined from the above line, which essentially
determines if enough time has passed to toggle the state of Valve
      digitalWrite(valve1, valve1State ? HIGH : LOW); // Enforcing the new state upon the valve physically
      previousValve1Millis = currentMillis; // Setting the current time as the previous time and the starting over
    }
  } else {
    digitalWrite(valve1, LOW); // Turn off valve 1 if no object detected by sensor 1
  }

  // Control valve 2 based on Sensor 2
```

```
    if (distance2 >= minDistance && distance2 <= maxDistance) {
      unsigned long currentMillis = millis(); // Current time since the start of the Arduino board program
      if (currentMillis - previousValve2Millis >= valveInterval) {
        valve2State = !valve2State; // Toggle valve 2 state determined from the above line, which essentially
determines if enough time has passed to toggle the state of Valve
        digitalWrite(valve2, valve2State ? HIGH : LOW); // Enforcing the new state upon the valve physically
        previousValve2Millis = currentMillis; // Setting the current time as the previous time and the starting over
      }
    } else {
      digitalWrite(valve2, LOW); // Turn off valve 2 if no object detected by sensor 2
    }

    // Small delay for stability
    delay(1);
  }

  // Function to measure distance using HC-SR04
  float measureDistance(int trigPin, int echoPin) {
    // Send a 10us pulse to trigger pin
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    // Read the echo pin
    long duration = pulseIn(echoPin, HIGH);

    // Calculate the distance in mm (duration * speed of sound / 2)
    float distance = (duration * 0.343) / 2.0;

    return distance;
  }
```

Model of the bread board in TinkerCad